

i wanted to create a website for a physical training institute named Purulia Aim Physical institute

they train youth for physical fitness and prepare for indian army , navy , airforce , police , rpf and all other government exams of india which requires to prove physical efficiency .

the trainer named Anup Kumar Mahato is an ex army Havaladar he is providing free physical training to all the youths of bengal .

what should be the contents i need to create

like there should be form to apply for membership , students profile so that they can get updated notifications , can track their growth ,

wall of fame

gallery

news and updates

contact

and admin dashboard from where the trainers can connect to the students and can see their growth

a blog site where students can share their motivating journey etc

Yes. For this institute, I would not treat it as just a website. It should be designed as a training + student management + community platform.

The key is to separate the system into modules before thinking about pages or UI.

Step 1: Define the core modules

I would structure Purulia Aim Physical Institute into these modules:

Module What it should contain

- | | |
|----------------------|---|
| 1. Home | Institute introduction, trainer, free-training mission, achievements, upcoming batches, CTA to join |
| 2. About Institute | History, mission, vision, training philosophy, facilities |
| 3. Trainer / Team | Anup Kumar Mahato's profile, ex-Army Havaladar background, experience, achievements |
| 4. Training Programs | Army, Navy, Air Force, Police, RPF, SSC GD, CAPF and other physical-test preparation |

5. Join / Membership Online application, student details, preferred training, physical measurements, documents, emergency contact
6. Student Portal Personal profile, batch, attendance, notifications, training schedule, progress, achievements
7. Physical Progress Tracking Running time, distance, push-ups, sit-ups, pull-ups, long jump, high jump, BMI, etc. with historical graphs
8. Attendance Daily attendance, attendance percentage, absence records
9. Performance / Tests Regular physical tests with scores and comparison against target standards
10. Notifications Batch announcements, test dates, recruitment notifications, training changes, holidays
11. Government Recruitment Updates Army/Navy/Air Force/Police/RPF/CAPF recruitment information and physical-test requirements
12. Wall of Fame Students selected for Army, Police, RPF, etc., with photo, name, achievement and year
13. Student Stories / Blog Students can publish their journey, struggles, training experience and success stories
14. Gallery Training photos, running sessions, camps, competitions, events
15. News & Events Institute activities, camps, competitions, selection celebrations
16. Testimonials Students and parents describing their experience
17. Contact Location, phone, WhatsApp, email, map, training timings
18. Admin Dashboard Manage students, batches, attendance, progress, notifications, gallery, blogs, achievements, etc.

But there is one important distinction.

Don't make the Student Portal just a profile page

The most valuable feature of this website can be a Student Progress System.

For example:

Student: Rahul Mahato

Batch: Morning Batch

Joined: 12 June 2026

PHYSICAL PROGRESS

	Initial	Current	Target

1600m Run	7:42	6:18	6:00
Push-ups	22	41	50
Sit-ups	28	48	50
Pull-ups	4	11	12
Long Jump	3.10m	3.65m	4.00m

Attendance: 91%

Tests

└─ June Test 62%

└─ July Test 71%

└─ August Test 78%

└─ September 84%

Then show the progress graphically.

The trainer should be able to enter these values from the admin panel, while the student can only view their own data.

One more module I strongly recommend

Add a Recruitment Target/Preparation module.

A student could select:

My Target

- ☒ Indian Army
- ☐ Indian Navy
- ☐ Indian Air Force
- ☒ West Bengal Police
- ☐ RPF

Then the system can show:

YOUR PREPARATION

Army

- └— Running ✓
- └— Push-ups ✓
- └— Physical Test 78%
- └— Documents Pending

WB Police

- └— Running ✓
- └— Physical Test 82%
- └— Recruitment Active

This makes the platform much more useful than a normal institute website.

Admin side should eventually have

ADMIN DASHBOARD

Students

- └— All Students
- └— Pending Applications
- └— Active Students

- └─ Inactive Students

- └─ Selected Students

Training

- └─ Batches

- └─ Timetable

- └─ Attendance

- └─ Physical Tests

- └─ Performance

Communication

- └─ Notifications

- └─ Announcements

- └─ Student Messages

Content

- └─ Gallery

- └─ News

- └─ Events

- └─ Blogs

- └─ Testimonials

- └─ Wall of Fame

Recruitment

- └─ Army

- └─ Navy

- └─ Air Force

- └─ Police

- └─ RPF

- └─ Other Forces

Reports

- └— Student Progress
- └— Attendance
- └— Batch Performance
- └— Selection Statistics

And because Anup Kumar Mahato is providing free physical training, I would make that a major part of the website's identity rather than hiding it in the About page.

For example:

Free Physical Training for the Youth of Bengal

Train with discipline. Build fitness. Prepare for your future.

The next thing we should decide is the complete website sitemap and user roles before discussing technologies or database design. That will prevent the project from becoming a collection of random features.

this is what i exactly want

now prepare me a detailed document with suitable tech stack (React/Nextjs), vanilla css or tailwind css , shadcn uis , proper authentication , fully secured , backend in node or java , database(sql or nosql) , folder structure , software architecture , and all major related things

i want this website to be a modern animated , graphics added

website that attracts youth as well as give a positive vibe of army with that much josh and enthusiasm and energy through its colors background styling and all

at the same time it should be professional also should cover each and every minute things that a physical institute requires to have

Purulia Aim Physical Institute

Technical & Product Architecture Document

Version: 1.0

Project Type: Training Institute Website + Student Management Platform + Community Platform

1. Project Vision

Purulia Aim Physical Institute (PAPI) should not be built as a conventional coaching-institute website.

It should be a digital platform for physical training, defence/police aspirants, student development, communication, recruitment preparation and community building.

The platform should communicate three things simultaneously:

1. Discipline

Military-inspired visual language, structured training, measurable progress and consistency.

2. Energy

Youth-oriented animations, motion graphics, powerful photography, dynamic statistics and energetic colors.

3. Trust

Professional typography, clean layouts, transparent information, secure authentication and reliable student management.

The website should make a visitor think:

"This is a serious physical-training institute, but it understands today's youth."

2. Recommended Technology Stack

My recommendation would be:

Layer	Technology
Frontend	Next.js + TypeScript
UI	Tailwind CSS
Component System	shadcn/ui
Animation	Framer Motion / Motion
Advanced animations	GSAP, where genuinely useful
Icons	Lucide React
Charts	Recharts
Forms	React Hook Form
Validation	Zod
Backend	Node.js + NestJS
API	REST API
Database	PostgreSQL
ORM	Prisma
Authentication	JWT + HttpOnly Secure Cookies
Password hashing	Argon2id
File storage	Cloudinary / S3-compatible storage
Email	Nodemailer / transactional email provider
SMS/OTP	Optional provider
Notifications	Database + Web Push/Firebase later
Maps	Google Maps / OpenStreetMap
Deployment	Vercel + Render/Railway/AWS
Reverse proxy	Nginx if self-hosting
Testing	Vitest/Jest + Playwright
API documentation	Swagger/OpenAPI
Monitoring	Sentry
CI/CD	GitHub Actions
Version control	Git + GitHub

3. Why Next.js?

Use Next.js, not plain React/Vite, for the public-facing website.

You need:

SEO

fast initial loading

server-side rendering

static generation

metadata management

social sharing previews

sitemap

robots.txt

image optimization

good performance

public recruitment/news/blog pages

For example:

/purulia-aim

/purulia-aim/about

/purulia-aim/training

/purulia-aim/recruitment

/purulia-aim/wall-of-fame

/purulia-aim/blog

/purulia-aim/contact

These pages need to be discoverable by Google.

A pure client-rendered application is unnecessary for this part.

4. Why Tailwind CSS?

I would choose:

Tailwind CSS > Vanilla CSS

Not because Vanilla CSS is bad.

Your project has a large number of UI states:

Public Website

Student Dashboard

Trainer Dashboard

Admin Dashboard

Forms

Tables

Charts

Cards

Modals

Mobile Navigation

Notifications

Authentication

Progress Components

Tailwind gives you consistent design primitives without creating hundreds of CSS files.

However:

Don't make the mistake of writing everything as giant Tailwind class strings.

Create reusable components:

Button

Card

SectionHeading

StatCard

ProgressCard

Modal

Input

Select

DataTable

Badge

Timeline

Then Tailwind remains manageable.

5. Why shadcn/ui?

Use shadcn/ui as the foundation for application components, not as the visual identity.

For example:

Dialog

DropdownMenu

Tabs

Sheet

Form

Input

Select

Table

Calendar

Toast

Alert

Tooltip

Then customize them to match PAPI's identity.

The public website should not look like a default shadow website.

That would make the site look like another developer dashboard.

6. Visual Design Direction

This is one of the most important parts of your project.

I would not make it completely military-themed.

That can quickly become tacky.

Instead:

Primary visual language

Army discipline

+

Youth energy

+

Modern sports aesthetics

+

Professional institution

7. Color System

A possible design system:

Primary

Deep Olive

#26351F

Secondary

Army Green

#4B6135

Energy

Saffron / Orange

#F59E0B

Accent

Warm Yellow

#FACC15

Dark background

#0B0F0A

Light background

#F5F7F2

Don't use green + orange + yellow everywhere.

Use:

Green = identity

Orange = action

Yellow = achievement

White = clarity

Dark = strength

8. Typography

Use a strong modern font combination.

For example:

Headings:

Bebas Neue / Oswald / Rajdhani

Body:

Inter / Manrope

But don't use military-style fonts for everything.

Example:

TRAIN HARD.

STAY DISCIPLINED.

SERVE THE NATION.

can use a display font.

Normal content:

Training schedules

Student information

Recruitment details

should use a highly readable sans-serif.

9. Hero Section

The homepage should immediately communicate the institute's purpose.

Possible structure:

PURULIA AIM PHYSICAL INSTITUTE

TRAIN • DISCIPLINE • ACHIEVE

Free Physical Training for the Youth of Bengal

[JOIN THE INSTITUTE] [EXPLORE TRAINING]

[Dynamic Training Video]

Behind the content:

running track

athletes running

sunrise

training silhouettes

push-ups

sprinting

group training

Use subtle animated particles/grid/lighting rather than excessive effects.

10. Homepage Structure

I would design the homepage approximately like this:

Hero



Institute Statistics



Why Choose PAPI



Training Programs



Meet The Trainer



Training Philosophy



Physical Progress System



Upcoming Recruitment Opportunities



Student Success Stories



Wall of Fame



Training Gallery



Student Journey



Latest Updates



Join The Institute



Contact / Location



Footer

11. Institute Statistics

Animated counters:

500+

Students Trained

100+

Successful Selections

10+

Years Training Experience

100%

Free Physical Training

Only display numbers that you can actually substantiate.

Don't invent impressive statistics for the design.

12. Training Programs

Create separate program cards.

Indian Army

Indian Navy

Indian Air Force

Police

RPF

CAPF

SSC GD

Other Physical Recruitment

Each program should contain:

Eligibility

Physical Requirements

Running Requirements

Strength Requirements

Height/Chest requirements where applicable

Selection Process

Preparation Strategy

Relevant recruitment links

Important:

Recruitment requirements change.

Therefore, don't hard-code them permanently into the application.

Build a recruitment-content management system.

13. Recruitment Information System

This can become one of the strongest features.

Example:

Indian Army

Recruitment: Active

Important Dates

Application:

Physical:

Written:

Medical:

Physical Requirements

Running:

Push-ups:

Pull-ups:

Height:

Chest:

Official Notification

[View Details]

Admin should be able to update this.

The database could store:

Recruitment

└— organization

└— title

└— notificationDate

└— applicationStart

└— applicationEnd

└— physicalDate

└— eligibility

└— physicalRequirements

└— officialUrl

└— status

14. Student Registration

The registration system should collect only information you actually need.

Basic information

Name

Date of Birth

Gender

Phone

Email

Address

District

Village

Emergency Contact

Physical information

Height

Weight

Chest

Blood Group (only if genuinely required)

Training information

Target

Preferred Batch

Previous Training

Current Running Time

Documents

Possible:

Photograph

Identity document

Certificates

Other relevant documents

Don't collect sensitive documents just because you can.

15. Student Application Workflow

Don't immediately create every applicant as an active student.

Use:

Application



Admin Review



Approved



Student Account Created



Batch Assigned



Training Begins

Application statuses:

PENDING

UNDER_REVIEW

APPROVED

REJECTED

WAITLISTED

16. Authentication Architecture

Use proper authentication.

Roles:

SUPER_ADMIN

ADMIN

TRAINER

STUDENT

CONTENT_MANAGER

Potentially later:

PARENT

17. Authentication Flow

Recommended:

Login



Backend validates credentials



Password verified with Argon2id



Access token generated



Refresh token generated



Tokens stored securely



HttpOnly Secure Cookie

Do not store authentication tokens in:

localStorage

for a sensitive student-management application.

18. Password Security

Never store:

password: "raj123"

or even encrypted passwords.

Store a strong password hash using:

Argon2id

Additional protection:

Rate limiting

Login attempt throttling

Password reset tokens

Email verification

Session management

Logout

Refresh-token rotation

19. Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

Example:

Student



Can see own profile

Can see own progress

Can see own attendance

Can receive notifications

Trainer



Can see assigned students

Can record attendance

Can record performance

Admin



Can manage students

Can manage batches

Can manage content

Super Admin



Can manage admins

Can manage permissions

Can access everything

This must be enforced on the backend, not merely by hiding buttons in React.

20. Backend Choice

For this project:

Node.js + NestJS

I would choose NestJS rather than raw Express.

Why?

Your backend will eventually contain:

Authentication

Students

Trainers

Batches

Attendance

Physical Tests

Progress

Recruitment

Notifications

Blogs

Gallery

Events

Reports

Admin

Audit Logs

File Uploads

Raw Express can handle it, but the architecture becomes your responsibility.

NestJS provides stronger structure around:

Modules

Controllers

Services

Guards

DTOs

Validation

Dependency Injection

Authentication

Authorization

This is especially useful for a project of this size.

21. Backend Architecture

Use a modular architecture.

API

|

├— Authentication

├— Users

├— Students

├— Trainers

├— Batches

├— Attendance

├— Assessments

├— Progress

├— Recruitment

- └─ Notifications
- └─ Blog
- └─ Gallery
- └─ Events
- └─ Testimonials
- └─ Wall of Fame
- └─ Files
- └─ Administration

Don't create one gigantic:

server.js

That becomes a maintenance nightmare.

22. Database

I strongly recommend:

PostgreSQL

rather than MongoDB for the core system.

Why?

Your data has relationships:

Student

↓

Batch

↓

Trainer

Student



Attendance

Student



Physical Tests



Performance

Student



Achievements

Student



Notifications

This is relational data.

PostgreSQL gives you:

foreign keys

transactions

constraints

indexes

joins

aggregation

strong consistency

reporting

This application is a much better fit for SQL than a document database.

23. Suggested Database Structure

High-level:

users

students

trainers

roles

permissions

student_profiles

student_targets

emergency_contacts

batches

batch_members

training_sessions

attendance

physical_tests

physical_test_results

progress_records

recruitments

recruitment_requirements

notifications

notification_reads

blogs

blog_categories

blog_comments

gallery

gallery_albums

events

event_registrations

achievements

wall_of_fame

testimonials

documents

files

audit_logs

24. Important Relationship

For example:

User

|

└─ StudentProfile

|

└─ Batch

└─ Targets

- └— Attendance
- └— Tests
- └— Progress
- └— Achievements
- └— Notifications

This gives you a clean domain model.

25. Physical Progress Engine

This deserves its own module.

Instead of simply storing:

```
running_time = "6:20"
```

store structured measurements.

Example:

PhysicalTestResult

studentId

testId

date

runningDistance

runningTimeSeconds

pushups

situps

pullups

longJumpCm

highJumpCm

weightKg

heightCm

Then you can calculate:

Improvement

Percentage change

Personal best

Target gap

Trend

26. Progress Dashboard

Student sees:

MY PROGRESS

1600M RUN



6:18

↓ 1:24 improvement

Push-ups

41

↑ +19

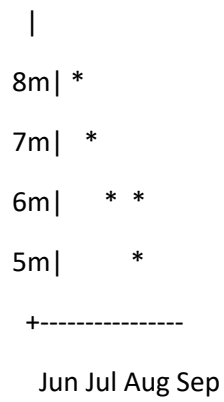
Pull-ups

11

↑ +7

And charts:

Running Time



This is where Recharts becomes useful.

27. Attendance System

Trainer dashboard:

Batch: Morning Batch

☒ Rahul

☒ Amit

☒ Suman

☐ Bikash

☒ Ankit

Save attendance.

Student:

Attendance

September: 94%

August: 91%

July: 88%

You can also calculate:

Present

Absent

Late

Leave

28. Training Session Management

Trainer should be able to create:

Morning Running Session

Date: 28 Sept

Time: 5:30 AM

Warm-up

Running

Sprint

Push-ups

Cool-down

Students receive the schedule.

29. Trainer Dashboard

Example:

GOOD MORNING, ANUP SIR

Today's Training

Morning Batch

45 Students

Attendance

38 / 45

Today's Session

Running + Strength

Performance Alerts

5 students haven't improved

3 students have missed 3 sessions

7 students achieved personal best

This is much more useful than a generic admin dashboard.

30. Student Communication

Don't immediately build a WhatsApp clone.

That's unnecessary.

Start with:

Notifications

Announcements

Training reminders

Personal messages

Example:

ANNOUNCEMENT

Tomorrow's morning training will start
at 5:15 AM instead of 5:30 AM.

Posted by Anup Kumar Mahato

Later you can add:

Trainer → Student chat

31. Blog / Student Journey

Students can submit:

My Journey

My First 5KM

How I Improved My Running

My Army Preparation

My Selection Story

Workflow:

Student writes



Submit



Admin moderation



Approved



Published

Never allow direct public publishing without moderation.

32. Wall of Fame

This should be emotionally powerful.

Example:

WALL OF FAME

"From the training ground
to the uniform."

[PHOTO]

Rahul Mahato

Selected for

Indian Army

2026

Read Journey →

Filters:

Army

Navy

Air Force

Police

RPF

CAPF

Other

33. Gallery

Don't simply create a grid of random images.

Use albums:

Training Sessions

Army Preparation

Running Camps

Competitions

Independence Day

Republic Day

Selection Celebrations

Community Activities

Special Events

Use optimized images.

34. Events

Example:

Upcoming

Physical Fitness Camp

5 October 2026

Purulia

[\[View Details\]](#)

Possible event types:

Training Camp

Fitness Test

Awareness Program

Competition

Selection Celebration

Community Event

35. Contact System

Don't only show:

Phone number

Address

Add:

Phone

WhatsApp

Email

Google Maps

Training location

Training timings

Emergency contact

Contact form:

Name

Phone

Email

Purpose

Message

Protect it against spam.

36. Admin CMS

The admin shouldn't need a developer for normal content changes.

Admin should be able to manage:

Homepage

About

Training Programs

Recruitment Updates

News

Events

Gallery

Blogs

Testimonials

Wall of Fame

FAQs

Contact Information

37. File Management

Images and documents should not be stored directly inside PostgreSQL.

Store files in:

Cloudinary

or:

AWS S3

Database stores:

fileId

url

publicId

type

owner

createdAt

For example:

Student



Profile Image



Cloudinary

38. Security Architecture

This needs to be taken seriously because you'll store student information.

Minimum security:

HTTPS

HttpOnly Cookies

Secure Cookies

SameSite protection

Argon2id

Rate Limiting

CORS

CSRF protection where applicable

Input Validation

Output Sanitization

RBAC

SQL Injection protection

XSS protection

File validation

File size limits

Audit logging

Security headers

Password reset security

Session invalidation

39. API Security

Every API endpoint should have a reason for existing.

Bad:

GET /api/students

where anyone can access everyone.

Better:

GET /api/students/me

Student sees own data.

Trainer:

GET /api/trainer/students

with backend authorization.

Admin:

GET /api/admin/students

with appropriate permissions.

40. Audit Logs

This is a feature many small systems forget.

Record important actions:

Who

What

When

IP

Resource

Action

Example:

Anup Kumar Mahato

Updated Rahul's 1600m performance

28 Sept 2026

10:32 AM

If something goes wrong, you can investigate it.

41. Frontend Folder Structure

Recommended:

frontend/

|

├─ app/

| └─ (public)/

| | └─ page.tsx

| | └─ about/

| | └─ training/

| | └─ recruitment/

| | └─ wall-of-fame/

| | └─ gallery/

| | └─ blog/

| | └─ events/

| | └─ contact/

| |

| └─ (auth)/

| | └─ login/

| | └─ register/

| | └─ forgot-password/

| |

- | └─ student/
- | | └─ dashboard/
- | | └─ profile/
- | | └─ progress/
- | | └─ attendance/
- | | └─ notifications/
- | | └─ training/
- | |
- | └─ trainer/
- | | └─ dashboard/
- | | └─ students/
- | | └─ attendance/
- | | └─ assessments/
- | |
- | └─ admin/
- | | └─ dashboard/
- | | └─ students/
- | | └─ batches/
- | | └─ recruitment/
- | | └─ content/
- | | └─ settings/
- |
- └─ components/
- | └─ ui/
- | └─ layout/
- | └─ navigation/
- | └─ hero/
- | └─ student/
- | └─ progress/
- | └─ recruitment/

```
|   └─ charts/
|
|   └─ lib/
|   └─ hooks/
|   └─ services/
|   └─ schemas/
|   └─ types/
|   └─ constants/
└─ public/
```

42. Backend Folder Structure

NestJS:

```
backend/
|
|   └─ src/
|   |
|   |   └─ auth/
|   |   |   └─ auth.controller.ts
|   |   |   └─ auth.service.ts
|   |   |   └─ auth.module.ts
|   |   |   └─ guards/
|   |   |   └─ strategies/
|   |   |
|   |   └─ users/
|   |   └─ students/
|   |   └─ trainers/
|   |   └─ batches/
|   |   └─ attendance/
|   |   └─ assessments/
```

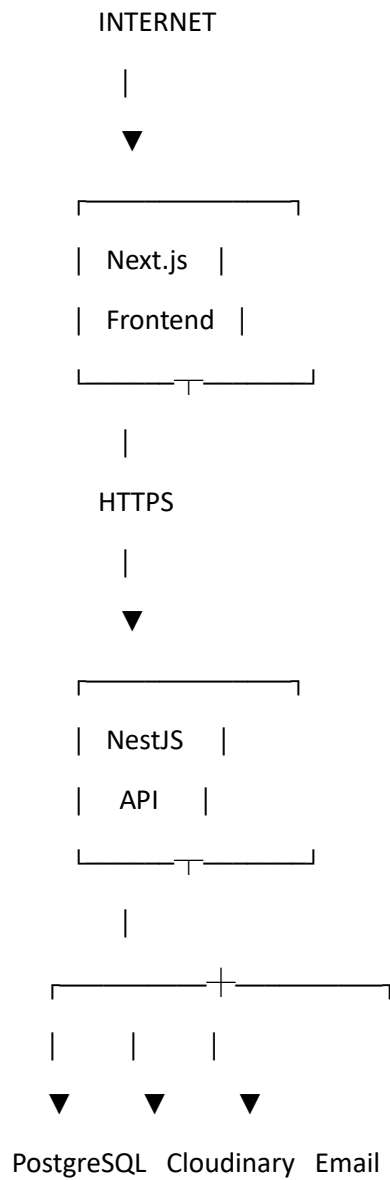
```
|   ├── progress/
|   ├── recruitment/
|   ├── notifications/
|   ├── blogs/
|   ├── gallery/
|   ├── events/
|   ├── testimonials/
|   ├── achievements/
|   ├── files/
|   ├── audit/
|   |
|   ├── common/
|   |   ├── guards/
|   |   ├── decorators/
|   |   ├── filters/
|   |   ├── interceptors/
|   |   ├── pipes/
|   |   └── middleware/
|   |
|   ├── prisma/
|   ├── config/
|   └── main.ts
|
└── prisma/
    ├── schema.prisma
    └── migrations/
|
└── test/
```

43. Architecture

I recommend a modular monolith initially.

Not microservices.

Architecture:



This is enough.

Don't introduce Kubernetes, Kafka, Redis clusters, microservices and other infrastructure just because they sound professional.

44. Repository Structure

I would use a monorepo:

```
purulia-aim/  
|  
├─ apps/  
|   ├─ web/  
|   └─ api/  
|  
├─ packages/  
|   ├─ types/  
|   ├─ validation/  
|   └─ config/  
|  
├─ docs/  
|   ├─ architecture/  
|   ├─ api/  
|   └─ database/  
|   └─ requirements/  
|  
├─ .github/  
|   └─ workflows/  
|  
├─ docker-compose.yml  
├─ package.json  
└─ README.md
```

This becomes particularly useful if frontend and backend share TypeScript types.

45. Development Environments

Have:

Development

Testing

Production

Never use production credentials locally.

Environment variables:

DATABASE_URL

JWT_SECRET

JWT_REFRESH_SECRET

CLOUDINARY_URL

SMTP_HOST

SMTP_USER

SMTP_PASSWORD

Secrets should never enter Git.

46. Testing Strategy

At minimum:

Unit tests

Test:

Authentication

Progress calculations

Attendance calculations

Recruitment logic

Permissions

API integration tests

Test:

Login

Student creation

Attendance

Progress updates

Authorization

End-to-end tests

Use Playwright for:

Student registration

Login

Student dashboard

Admin login

Creating a batch

Recording attendance

Updating progress

47. SEO

Public pages should be SEO optimized.

Implement:

Metadata

OpenGraph

Twitter cards

Sitemap

Robots.txt

Canonical URLs

Structured data

Potential search terms:

Physical training institute Purulia

Army physical training Purulia

Army preparation Purulia

Police physical training Purulia

RPF physical training Purulia

Free physical training Bengal

Don't keyword-stuff the website.

48. Performance

Because you're using animations and graphics, performance becomes important.

Use:

Next Image

WebP/AVIF

Lazy loading

Code splitting

Dynamic imports

Compressed videos

Optimized fonts

Caching

CDN

Don't put a 20 MB video in the hero and call it "modern."

A slow website destroys the exact youth experience you're trying to create.

49. Animation Strategy

Use animation with purpose.

Hero

Fade + slide

Background movement

Subtle particles

Text reveal

Statistics

Counter animation

Training cards

Hover

Image zoom

Border transition

Progress

Animated chart

Wall of Fame

Image reveal

Avoid:

Everything bouncing

Everything spinning

Text flying everywhere

Constant background animations

The website should feel energetic, not childish.

50. Advanced Visual Features

Later you can add:

Scroll-based storytelling

TRAIN



DISCIPLINE



PERSIST



ACHIEVE



SERVE

Training timeline

Day 1



Basic Fitness



Endurance



Strength



Speed



Physical Test



Selection

This could become one of the signature visual sections.

51. Mobile-First Design

This is critical.

Many students will access this using:

Android phone

Mobile data

Chrome

Therefore:

Mobile → Tablet → Desktop

not the reverse.

Student dashboard must work perfectly on a phone.

52. PWA

Eventually consider making the student portal a Progressive Web App.

Then students can install:

Purulia Aim

on their phone.

They could receive:

Training reminder

New announcement

Test result

Attendance warning

Achievement

without needing a native Android application initially.

53. Notification Architecture

Database:

notifications

notification_recipients

notification_reads

Types:

GENERAL

BATCH

PERSONAL

TRAINING

RECRUITMENT

RESULT

SYSTEM

Later:

Push Notification

Email

WhatsApp

SMS

can be added.

54. Reports

Admin should eventually be able to generate:

Student Performance Report

Attendance Report

Batch Performance

Monthly Progress

Selection Report

Training Activity

PDF:

PURULIA AIM PHYSICAL INSTITUTE

Student Performance Report

Student:

Batch:

Training Period:

Running:

Push-ups:

Pull-ups:

Attendance:

Trainer Remarks:

Trainer Signature

55. Trainer Remarks

This is an underrated feature.

After a test:

Performance:

Good

Trainer Remark:

"Running has improved significantly.

Work on sprint acceleration and recovery."

Students will feel that the system is actually connected to their training.

56. Student Achievement System

Add badges:

5KM Completed

10KM Completed

Personal Best

30 Days Consistency

90% Attendance

100 Push-ups Challenge

First Physical Test

Selected Candidate

This can make the student portal more engaging.

But avoid turning physical training into a meaningless gaming system.

Achievements should represent real accomplishments.

57. Community Features

Later:

Student Stories

Training Challenges

Success Stories

Motivational Posts

Events

Achievements

But all public content should have moderation.

58. FAQ

Create a proper FAQ:

Is training really free?

Who can join?

What age group can participate?

Where is training conducted?

What should I bring?

What are the training timings?

Which recruitments do you prepare for?

Is accommodation available?

Do you provide recruitment guidance?

How do I apply?

This reduces repetitive phone calls.

59. Legal Pages

Don't forget:

Privacy Policy

Terms of Use

Cookie Policy

Student Data Policy

Photo/Media Consent

Disclaimer

Especially because you will publish student photographs and stories.

For minors, parental/guardian consent may be required depending on the circumstances.

60. Photo Consent

When students register:

☐ I agree to the institute using my photograph
for official website/social media purposes.

Better still, separate:

Training system consent

Publicity/media consent

Don't bundle everything into one vague checkbox.

61. Admin Security

Admin panel should have additional protection.

Consider:

2FA

Login rate limiting

Session expiry

Device/session management

Audit logs

Strong password policy

Admin IP alerts

2FA is especially worthwhile for Super Admin accounts.

62. Backup Strategy

PostgreSQL:

Daily automated backup

Weekly retained backup

Point-in-time recovery where supported

Files:

Cloud storage

Versioning where appropriate

Never consider:

"The database provider handles backups."

as a complete disaster-recovery strategy.

You need to know:

How do we restore the system if the database disappears tomorrow?

63. Disaster Recovery

Document:

Database restoration

File restoration

Environment recreation

Admin credential recovery

DNS recovery

Deployment recovery

This belongs in your technical documentation.

64. Analytics

Add privacy-conscious analytics.

Track:

Visitors

Popular pages

Recruitment page visits

Join-form conversions

Contact requests

Blog views

Don't track unnecessary personal student information.

65. Admin Dashboard Main Screen

A useful dashboard might look like:

GOOD MORNING, ANUP SIR

Active Students	247
Today's Attendance	218
Pending Applications	18
Upcoming Tests	3
Attendance Trend	
	
Students Needing Attention	
Rahul	↓ Running performance
Amit	3 days absent
Suman	Personal best

66. User Roles

Final role architecture:

SUPER_ADMIN

|

|— ADMIN

|

|— TRAINER

|

|— CONTENT_MANAGER

|
└─ STUDENT

Don't give every employee full admin privileges.

Follow least privilege.

67. Suggested MVP

This project is too large to build all at once.

Build Version 1 first.

Phase 1: Public Website

Home

About

Trainer

Training Programs

Recruitment

Wall of Fame

Gallery

News

Blog

Contact

Join

Phase 2: Authentication

Registration

Login

Forgot Password

Student Profile

Roles

Phase 3: Student Portal

Dashboard

Attendance

Progress

Physical Tests

Notifications

Training Schedule

Phase 4: Trainer/Admin

Students

Batches

Attendance

Physical Tests

Progress

Notifications

Phase 5: CMS

Blog

Gallery

Events

Wall of Fame

Recruitment Updates

Testimonials

Phase 6: Advanced

Reports

PDFs

Push Notifications

PWA

Analytics

Advanced recruitment tracking

68. What I Would NOT Build Initially

Avoid these in V1:

Native Android app

Native iOS app

Microservices

Real-time chat

Payment gateway

AI fitness coach

Blockchain

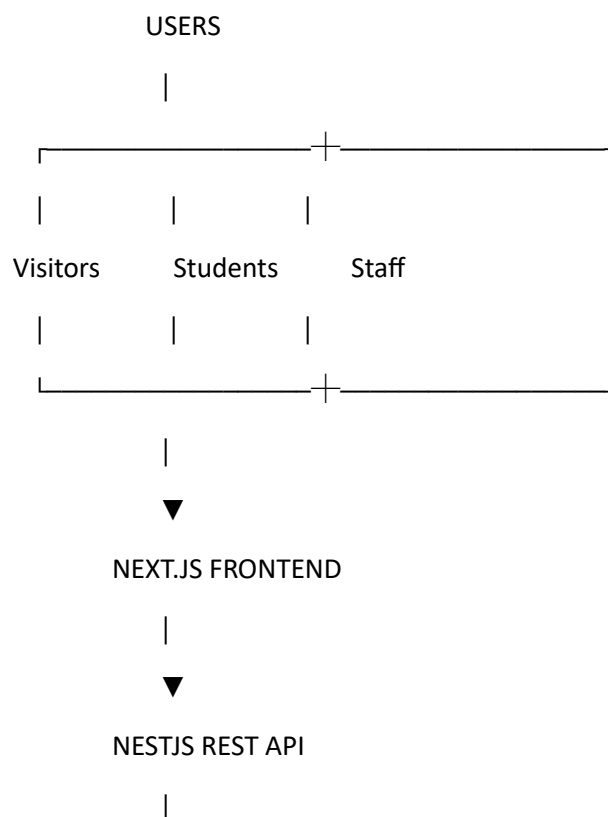
Complex social network

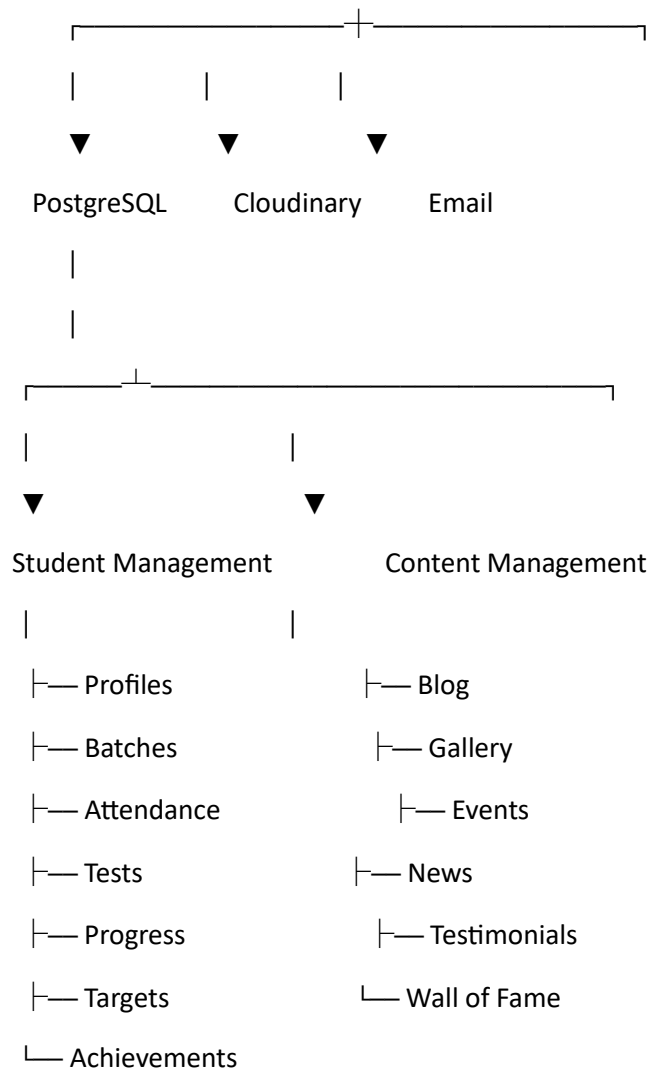
Live GPS tracking

They add complexity without proving whether students actually need them.

69. Final Architecture

The complete system eventually looks like:





70. Recommended Final Stack

If I were building this project with you, I would lock the stack as:

Frontend

Next.js

TypeScript

Tailwind CSS

shadcn/ui

Motion / Framer Motion

GSAP where required

Recharts

React Hook Form

Zod

Backend

Node.js

NestJS

TypeScript

REST API

Swagger/OpenAPI

Database

PostgreSQL

Prisma ORM

Authentication

JWT

HttpOnly Secure Cookies

Argon2id

RBAC

Refresh Token Rotation

2FA for Admin

Storage

Cloudinary

Infrastructure

Vercel

Render/Railway/AWS

GitHub

GitHub Actions

Testing

Vitest/Jest

Playwright

Monitoring

Sentry

Design

Army Green

Olive

Saffron

Yellow

Black

White

Modern typography

Sports photography

Motion graphics

One architectural decision I would strongly insist on

Don't build this as "a website with an admin panel."

Build it as:

Public Website + Training Management System + Student Portal + Recruitment Information Platform
+ Community

That distinction will affect your database, APIs, authentication, permissions and folder structure from day one.

And because you want this to be a serious portfolio/project-quality system, the next step should be requirements engineering, not coding: we should turn every feature above into actors → use cases → permissions → pages → API responsibilities → database entities. That will give us the actual blueprint before you write a single component.

ManageEngine

Malware Protection Built for SMBs.

Stop malware and ransomware with certified, multi-layered protection. Start your 30-day free trial.

Ad

More options

Software Requirements Specification

Purulia Aim Physical Institute (PAPI) Platform

Version: 1.0 | Date: 28 September 2026 | Status: Draft for review Project type: Public website + Student management system + Recruitment information platform + Community platform

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional, data, security and architectural requirements for the PAPI platform. It is the blueprint from which design, database schema, APIs and code will be derived.

1.2 Background

Purulia Aim Physical Institute trains youth in physical fitness and prepares them for the physical tests of the Indian Army, Navy, Air Force, Police, RPF, CAPF, SSC GD and similar government recruitments. The trainer, Anup Kumar Mahato, is an ex-Army Havaldar and provides free physical training to the youth of Bengal.

1.3 Product vision

Not a "website with an admin panel", but:

Public Website + Training Management System + Student Portal + Recruitment Information Platform
+ Community

The platform must communicate Discipline (military-inspired structure, measurable progress), Energy (motion, bold visuals, youth appeal) and Trust (professional layout, transparent data, secure accounts).

1.4 Scope

In scope (V1): public site, membership application, authentication and roles, student portal, attendance, physical tests and progress tracking, recruitment targets and updates, notifications, trainer/admin dashboards, CMS (news, events, gallery, wall of fame, testimonials, FAQ), moderated student blog, contact form, audit logs. Out of scope (V1): native mobile apps, real-time chat, payment gateway, AI coach, live GPS tracking, microservices, social network features. (Reports/PDF, push notifications and PWA are planned for Phase 6.)

1.5 Definitions

Term	Meaning
------	---------

PAPI	Purulia Aim Physical Institute
------	--------------------------------

RBAC	Role-based access control
------	---------------------------

CMS	Content management system (admin-managed content)
-----	---

Batch	A group of students training at a given time (e.g., Morning Batch)
-------	--

Assessment	A scheduled physical test event; results are per student
------------	--

Target	The recruitment a student is preparing for (Army, WB Police, etc.)
--------	--

PWA	Progressive Web App
-----	---------------------

2. Overall Description

2.1 Product perspective

Standalone web system: a Next.js frontend consuming a NestJS REST API backed by PostgreSQL, with external services for file storage (Cloudinary/S3) and email.

2.2 User classes

Role	Description
Visitor	Unauthenticated public user; reads content, submits application/contact form
Applicant	Visitor who has submitted a membership application (no login until approved)
Student	Approved member; views own profile, progress, attendance, notifications; submits stories
Trainer	Manages assigned batches; records attendance, tests, remarks
Content Manager	Manages news, events, gallery, blog moderation, wall of fame
Admin	Manages students, batches, applications, recruitment data, notifications
Super Admin	Full access; manages admins, roles, system settings. Mandatory 2FA
Parent (future)	Read-only view of a minor student's progress

2.3 Operating environment

Modern browsers (Chrome-first; many users are on Android phones over mobile data). Responsive from 360 px width upward. Server: Node.js LTS on a cloud host; PostgreSQL 15+.

2.4 Constraints

Mobile-first design and low-bandwidth tolerance.

Handles personal data of students, including minors: consent and data-minimisation are mandatory.

Training is free: no payment features in V1.

Small team: modular monolith, no over-engineered infrastructure.

2.5 Assumptions and dependencies

Recruitment rules change often, so they are stored as admin-editable data, never hard-coded.

Only verifiable statistics are displayed on the homepage; counters read from real data or admin-set values.

Third-party services available: Cloudinary/S3, email provider, maps.

3. Functional Requirements

Priority: M = Must (V1), S = Should, C = Could (later).

3.1 Public website

ID	Requirement	P
FR-PUB-01	Homepage with hero (tagline, CTAs "Join the Institute" / "Explore Training", optimised background video or image), animated statistics, why-choose-us, programs, trainer profile, training philosophy, progress-system showcase, upcoming recruitments, success stories, wall of fame preview, gallery preview, latest updates, join CTA, contact/map, footer	M

FR-PUB-02	Highlight "Free Physical Training for the Youth of Bengal" as core identity on the homepage, not only on About	M
FR-PUB-03	About page: history, mission, vision, philosophy, facilities	M
FR-PUB-04	Trainer page: Anup Kumar Mahato's profile, ex-Army Havaldar background, experience, achievements; supports additional trainers	M
FR-PUB-05	Training Programs: one page per program (Army, Navy, Air Force, Police, RPF, CAPF, SSC GD, Other) showing eligibility, physical requirements (running, strength, height/chest), selection process, preparation strategy, official links	M
FR-PUB-06	Recruitment Updates listing and detail pages with status (Active/Upcoming/Closed), key dates, requirements, official notification link	M
FR-PUB-07	Wall of Fame with photo, name, force, year, journey link; filter by force	M
FR-PUB-08	Gallery organised in albums (training sessions, camps, competitions, national days, selection celebrations, events); lightbox viewer	M
FR-PUB-09	News & Events pages; events have type, date, venue, details	M
FR-PUB-10	Testimonials from students and parents (moderated)	S
FR-PUB-11	Student Stories / Blog: list, detail, categories, search	M
FR-PUB-12	FAQ page (editable)	S
FR-PUB-13	Contact page: phone, WhatsApp, email, embedded map, training location and timings, spam-protected contact form	M
FR-PUB-14	Legal pages: Privacy Policy, Terms, Cookie Policy, Student Data Policy, Photo/Media Consent, Disclaimer	M
FR-PUB-15	Full SEO: metadata, OpenGraph/Twitter cards, sitemap.xml, robots.txt, canonical URLs, structured data	M

3.2 Membership application

ID	Requirement	P
FR-APP-01	Multi-step online form collecting only needed data: basic (name, DOB, gender, phone, email, address, district, village), emergency contact, physical (height, weight, chest), training (target force(s), preferred batch, previous training, current running time), optional documents (photo, ID, certificates)	M
FR-APP-02	Separate consent checkboxes: (a) training/data processing, (b) photo/publicity use; guardian consent captured when applicant is under 18	M
FR-APP-03	Server-side validation (Zod schemas shared with client); bot protection on submission	M
FR-APP-04	Application statuses: PENDING → UNDER_REVIEW → APPROVED / REJECTED / WAITLISTED with admin notes and timestamps	M
FR-APP-05	Approval creates the student account, sends set-password email, and allows batch assignment; rejection/waitlist sends email	M

FR-APP-06 Applicants can be searched, filtered and exported by admin S

3.3 Authentication and authorization

ID Requirement P

FR-AUTH-01 Login by email/phone + password; logout; forgot/reset password via single-use expiring token M

FR-AUTH-02 Password hashing with Argon2id; strong password policy M

FR-AUTH-03 Short-lived access token + rotating refresh token, both in HttpOnly, Secure, SameSite cookies; no tokens in localStorage M

FR-AUTH-04 Email verification; login throttling and account logout M

FR-AUTH-05 RBAC enforced by backend guards on every endpoint (see §4) M

FR-AUTH-06 Active session list and remote session revoke; refresh-token reuse detection S

FR-AUTH-07 2FA (TOTP) mandatory for Super Admin, available for Admin/Trainer M

3.4 Student portal

ID Requirement P

FR-STU-01 Dashboard: batch, joined date, attendance %, latest test summary, next session, unread notifications, badges M

FR-STU-02 Profile view/edit (limited fields; sensitive fields edit via admin request) M

FR-STU-03 Progress page (see §3.5) M

FR-STU-04 Attendance page: monthly percentages, present/absent/late/leave history M

FR-STU-05 Training schedule: upcoming sessions with plan (warm-up, run, strength, cool-down) M

FR-STU-06 Notification centre with read/unread state M

FR-STU-07 My Targets: select forces (Army, Navy, Air Force, WB Police, RPF, CAPF, etc.) and view a per-target readiness checklist (running ✓, push-ups ✓, physical-test score %, documents status, recruitment status) M

FR-STU-08 Write and submit stories (draft → submitted → moderation) M

FR-STU-09 Students can only see their own data M

FR-STU-10 Installable PWA with training reminders C

3.5 Physical progress tracking engine

ID Requirement P

FR-PRG-01 Trainers define assessments (name, date, batch, metrics included) M

FR-PRG-02 Store structured numeric results per student per assessment: 1600 m run time (seconds), distance, push-ups, sit-ups, pull-ups, long jump (cm), high jump (cm), weight (kg), height (cm), BMI derived M

FR-PRG-03	Show Initial / Current / Target for each metric, improvement (absolute and %), personal best, gap to target, trend	M
FR-PRG-04	Charts (line/bar, e.g., running time over months, monthly test score %) using Recharts	M
FR-PRG-05	Target standards per force/metric configurable by admin (used for score % and readiness)	M
FR-PRG-06	Trainer remarks and performance rating per assessment	M
FR-PRG-07	Badges tied to real achievements (5 KM done, personal best, 30-day consistency, 90% attendance, first physical test, selected candidate)	S

Illustrative student view:

Metric	Initial	Current	Target
1600 m run	7:42	6:18	6:00
Push-ups	22	41	50
Sit-ups	28	48	50
Pull-ups	4	11	12
Long jump	3.10 m	3.65 m	4.00 m

3.6 Attendance and training sessions

ID	Requirement	P
FR-ATT-01	Trainer creates sessions (title, date, time, batch, plan)	M
FR-ATT-02	Mark attendance per session: Present / Absent / Late / Leave; bulk mark and edit within a configurable window	M
FR-ATT-03	Auto-calculate attendance % per student, batch and month	M
FR-ATT-04	Flag students with consecutive absences (configurable threshold)	S

3.7 Recruitment information system (CMS-driven)

ID	Requirement	P
FR-REC-01	Admin CRUD for recruitment entries: organization, title, notification date, application start/end, physical/written/medical dates, eligibility, physical requirements, official URL, status	M
FR-REC-02	Recruitment entries appear on public pages and are linkable to student targets	M
FR-REC-03	Notify students whose target matches a new or updated recruitment	S

3.8 Notifications and communication

ID	Requirement	P
FR-NOT-01	Types: GENERAL, BATCH, PERSONAL, TRAINING, RECRUITMENT, RESULT, SYSTEM	M

FR-NOT-02	Send to all, a batch, or an individual; store recipients and read receipts	M
FR-NOT-03	Announcements shown on public site when marked public	S
FR-NOT-04	Email delivery; Web Push, SMS/WhatsApp as future channels	S/C
FR-NOT-05	Trainer-to-student one-to-one messages (non-real-time)	C

3.9 Trainer dashboard

ID	Requirement	P
FR-TRN-01	Today's overview: batches, expected vs. marked attendance, today's session	M
FR-TRN-02	Assigned students list with progress snapshot and history	M
FR-TRN-03	Attention panel: no improvement, repeated absences, new personal bests	S
FR-TRN-04	Record assessments and remarks; send batch or personal notifications	M

3.10 Admin dashboard

ID	Requirement	P
FR-ADM-01	Overview cards: active students, today's attendance, pending applications, upcoming tests; attendance trend chart	M
FR-ADM-02	Students: all, pending applications, active, inactive, selected; view/edit/deactivate	M
FR-ADM-03	Batches and timetable management; assign trainers and students	M
FR-ADM-04	Manage users, roles and permissions (Super Admin)	M
FR-ADM-05	Communication: notifications, announcements	M
FR-ADM-06	Content: gallery, news, events, blogs, testimonials, wall of fame, FAQ, homepage/contact info	M
FR-ADM-07	Recruitment management (see §3.7)	M
FR-ADM-08	Reports: student progress, attendance, batch performance, monthly progress, selection statistics; PDF export with trainer remarks	S
FR-ADM-09	Audit log viewer (who, what, when, IP, resource, action)	M
FR-ADM-10	Mark a student as "Selected" and publish to Wall of Fame with consent check	M

3.11 Student stories / blog

ID	Requirement	P
FR-BLG-01	Students draft and submit stories with images	M
FR-BLG-02	Moderation workflow: DRAFT → SUBMITTED → APPROVED/REJECTED → PUBLISHED; nothing goes public without approval	M
FR-BLG-03	Categories/tags, sanitised rich text, reading time, share buttons	M
FR-BLG-04	Comments (moderated)	C

3.12 Files

ID	Requirement	P
FR-FIL-01	Files stored in Cloudinary/S3; DB stores metadata only (id, url, publicId, type, owner, createdAt)	M
FR-FIL-02	Validate type, size and content; private documents served by signed URLs	M

4. Roles and Permission Matrix

Least privilege; enforced server-side. R = read, W = write, O = own only, A = assigned only.

Resource	StudentTrainer	Content Mgr	Admin	Super Admin
Own profile	R/W(O)	R/W(O)	R/W(O)	R/W(O)
Student records	R(O)	R(A)	none	R/W
Applications	none	none	none	R/W
Batches / sessions	R(O)	R/W(A)	none	R/W
Attendance	R(O)	R/W(A)	none	R/W
Assessments / results	R(O)	R/W(A)	none	R/W
Recruitment data	R	R	R/W	R/W
Notifications	R(O)	W (own batches)	none	R/W
Blog / stories	R/W(O drafts)	R	R/W (moderate)	R/W
Gallery / news / events / wall of fame		R	R	R/W
Users / roles / settings	none	none	none	limited R/W
Audit logs	none	none	R	R

5. External Interface Requirements

User interface: responsive, mobile-first; consistent component library (shadcn/ui customised to PAPI identity); accessible navigation; separate layouts for public, student, trainer and admin areas. Software interfaces: PostgreSQL (via Prisma), Cloudinary/S3 (media), SMTP/transactional email provider, Google Maps or OpenStreetMap, Sentry (errors), optional Firebase/Web Push, optional SMS/OTP provider. Communication: HTTPS/TLS only; REST over JSON; cookies for auth; OpenAPI (Swagger) contract.

6. Non-Functional Requirements

Category	Requirement
Performance	LCP < 2.5 s on 4G mid-range Android for key public pages; hero video compressed (target under ~3 MB) with poster fallback; images WebP/AVIF via Next Image; lazy loading; code splitting; CDN caching; API p95 < 500 ms for standard reads

Security HTTPS/HSTS; Argon2id; HttpOnly+Secure+SameSite cookies; CSRF protection; CORS allow-list; rate limiting; input validation (DTO/Zod) and output sanitisation; parameterised queries via ORM; security headers (CSP, X-Frame-Options, etc.); file-upload validation and limits; RBAC on every endpoint; secrets in environment variables only, never in Git

Privacy and compliance Data minimisation; explicit, separate consents (training vs. publicity); guardian consent for minors; retention and deletion policy; consent required before publishing photos/stories; comply with applicable Indian data-protection law (to be confirmed with legal advice)

Availability and recovery Target 99.5% uptime; automated daily DB backups, weekly retained copies, point-in-time recovery where available; tested restore procedure; documented disaster recovery (DB, files, environment, DNS, admin recovery)

Usability Mobile-first; simple forms with clear errors; Bengali/English bilingual readiness (i18n structure from the start, translations phased)

Accessibility WCAG 2.1 AA target: contrast, keyboard navigation, alt text, prefers-reduced-motion respected

SEO Server-rendered/static public pages; semantic HTML; structured data; local-search terms (Purulia, physical training, Army/Police/RPF preparation) without keyword stuffing

Maintainability TypeScript throughout; modular monolith; shared types/validation package; linting, formatting, conventional commits; documentation in /docs

Scalability Sized for a few thousand students; stateless API enabling horizontal scale; DB indexes on foreign keys and query paths

Observability Sentry for errors; structured logs; health-check endpoint; privacy-conscious analytics (visitors, popular pages, join-form conversions)

Auditability Immutable audit log for sensitive actions (login, role changes, test edits, approvals, deletions)

7. Architecture

7.1 Style

Modular monolith (no microservices, Kubernetes or Kafka in V1).

Visitors / Students / Staff

|

HTTPS (CDN)

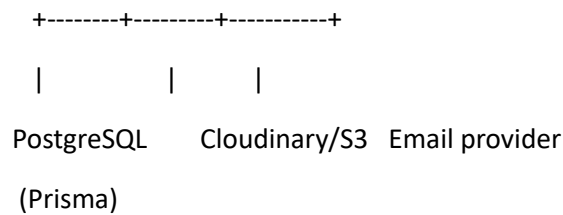
|

Next.js Frontend (SSR/SSG + client dashboards)

|

NestJS REST API (Guards, DTO validation, Swagger)

|



7.2 Technology stack

Layer Choice Rationale

Frontend framework Next.js + TypeScript SEO, SSR/SSG, metadata, image optimisation for public pages

Styling Tailwind CSS Consistent design tokens across many UI states; wrap in reusable components

Components shadcn/ui (Radix-based) Accessible primitives for dialogs, tables, forms; heavily themed, not default look

Animation Motion (Framer Motion); GSAP only for scroll storytelling Purposeful animation

Charts / icons Recharts / Lucide React Progress graphs, consistent icons

Forms / validation React Hook Form + Zod (shared package) Same rules client and server

Backend Node.js + NestJS (REST) Modules, guards, DTOs, DI suit a domain-heavy app; Swagger/OpenAPI

Database PostgreSQL Relational data: students, batches, attendance, tests; constraints, transactions, reporting

ORM Prisma Type-safe schema and migrations

Auth JWT access + rotating refresh in HttpOnly cookies, Argon2id, RBAC, TOTP 2FA See §3.3

Storage Cloudinary or S3-compatible Images, documents

Email Nodemailer / transactional provider Verification, resets, notifications

Testing Vitest/Jest, Playwright Unit, integration, E2E

Monitoring Sentry Errors and performance

CI/CD GitHub Actions Lint, test, build, deploy

Hosting Vercel (web) + Render/Railway/AWS (API, DB) Simple, low-ops; Nginx if self-hosted

Alternative considered: Java (Spring Boot) for the backend is viable but adds a second language; NestJS keeps one TypeScript stack with shared types. MongoDB was rejected for the core because data is strongly relational.

7.3 Environments

Development, Testing/Staging, Production; separate credentials; .env files never committed.
Variables include DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, CLOUDINARY_URL, SMTP_.*.

8. Data Requirements

8.1 Core entities

Group Tables

Identity users, roles, permissions, user_roles, sessions/refresh_tokens, password_reset_tokens, two_factor_secrets

Students applications, students, student_profiles, emergency_contacts, student_targets, consents, documents

Training batches, batch_members, batch_trainers, training_sessions, attendance

Performance assessments, assessment_results, target_standards, progress_records, trainer_remarks, badges, student_badges

Recruitment recruitments, recruitment_requirements

Communication notifications, notification_recipients, notification_reads, contact_messages

Content blogs, blog_categories, blog_comments, gallery_albums, gallery_items, events, event_registrations, news, testimonials, wall_of_fame, faqs, site_settings

System files, audit_logs

8.2 Key relationships

User 1—1 StudentProfile; Student N—N Batch (via batch_members); Batch 1—N TrainingSession; TrainingSession 1—N Attendance; Assessment 1—N AssessmentResult; Student 1—N AssessmentResult / Attendance / Target / Achievement; Notification 1—N Recipient; Recruitment 1—N Requirement.

8.3 Assessment result (structured, not free text)

studentId, assessmentId, date, runDistanceM, runTimeSeconds, pushups, situps, pullups, longJumpCm, highJumpCm, weightKg, heightCm, score, trainerRemark
Derived values (improvement, % change, personal best, target gap, BMI, trend) are computed by the API.

8.4 Enumerations

Role, ApplicationStatus (PENDING, UNDER_REVIEW, APPROVED, REJECTED, WAITLISTED), AttendanceStatus (PRESENT, ABSENT, LATE, LEAVE), NotificationType, ContentStatus (DRAFT, SUBMITTED, APPROVED, REJECTED, PUBLISHED), RecruitmentStatus (UPCOMING, ACTIVE, CLOSED), Force (ARMY, NAVY, AIR_FORCE, POLICE, RPF, CAPF, SSC_GD, OTHER).

8.5 Data rules

Foreign keys and unique constraints (one attendance row per student per session); soft-delete for students and content; no files or blobs in PostgreSQL; personal identifiers minimised and access-controlled.

9. API Outline (REST, versioned /api/v1)

Principle: every endpoint has a defined actor and scope, for example GET /students/me rather than an open GET /students.

Area Representative endpoints

Auth POST /auth/login, /auth/logout, /auth/refresh, /auth/forgot-password, /auth/reset-password, /auth/2fa/verify

Public GET /public/recruitments, /public/wall-of-fame, /public/gallery, /public/blogs, /public/events, POST /public/applications, POST /public/contact

Student GET /students/me, /students/me/progress, /students/me/attendance, /students/me/notifications, PUT /students/me/targets, POST /students/me/stories

Trainer GET /trainer/batches, /trainer/students, POST /trainer/sessions, POST /trainer/attendance, POST /trainer/assessments/:id/results

Admin GET/PATCH /admin/applications/:id, /admin/students, /admin/batches, /admin/users, /admin/recruitments, /admin/notifications, /admin/reports/*, /admin/audit-logs

Content/admin/content/{news,events,gallery,blogs,testimonials,wall-of-fame,faqs}

Conventions: DTO validation on all inputs, consistent error format, pagination and filtering, OpenAPI docs at /api/docs (restricted in production).

10. Key Workflows

Application: Visitor submits form → status PENDING → admin review → APPROVED (account created, set-password email, batch assigned) / REJECTED / WAITLISTED → training begins.

Assessment: Trainer creates assessment → enters results per student → system computes progress → student notified → trainer remark visible. Story moderation: Student drafts → submits → Content Manager/Admin approves or rejects with reason → published on public blog. Wall of Fame: Admin marks student as selected → consent verified → entry drafted → published with photo, force, year.

11. Repository and Folder Structure

purulia-aim/

└─ apps/

| └─ web/ # Next.js

| | └─ app/

- | | | └─ (public)/ # home, about, training, recruitment, wall-of-fame, gallery, blog, events, contact, join
- | | | └─ (auth)/ # login, forgot-password
- | | | └─ student/ # dashboard, profile, progress, attendance, notifications, training
- | | | └─ trainer/ # dashboard, students, attendance, assessments
- | | | └─ admin/ # dashboard, students, batches, recruitment, content, settings
- | | └─ components/ # ui/, layout/, hero/, student/, progress/, recruitment/, charts/
- | | └─ lib/ hooks/ services/ schemas/ constants/ public/
- | └─ api/ # NestJS
- | └─ src/
- | | └─ auth/ users/ students/ trainers/ batches/ attendance/
- | | └─ assessments/ progress/ recruitment/ notifications/
- | | └─ blogs/ gallery/ events/ testimonials/ achievements/
- | | └─ files/ audit/ common/ (guards, decorators, filters, interceptors, pipes)
- | | └─ prisma/ config/ main.ts
- | └─ prisma/ (schema.prisma, migrations/)
- └─ packages/ types/ validation/ config/
- └─ docs/ architecture/ api/ database/ requirements/
- └─ .github/workflows/
- └─ docker-compose.yml
- └─ README.md

12. Design and Experience Requirements

Direction: Army discipline + youth energy + modern sports aesthetics + professional institution. Avoid a tacky "costume" military look and avoid the default shadcn look.

Token	Value	Use
Deep Olive	#26351F	Primary identity
Army Green	#4B6135	Secondary
Saffron	#F59E0B	Actions / CTAs
Warm Yellow	#FACC15	Achievements
Dark	#0B0F0A	Strength, dark sections

Light #F5F7F2 Backgrounds

Typography: display font (Bebas Neue / Oswald / Rajdhani) for slogans such as "TRAIN HARD. STAY DISCIPLINED. SERVE THE NATION."; Inter or Manrope for body and data. Motion: hero fade/slide and text reveal; counter animations; card hover and image zoom; animated progress charts; image-reveal on Wall of Fame; optional scroll storytelling (Train → Discipline → Persist → Achieve → Serve) and a training-journey timeline. No constant or gimmicky motion; honour prefers-reduced-motion. Imagery: real training photography (running tracks, sunrise, group drills), not stock clichés; compressed media. Reusable components: Button, Card, SectionHeading, StatCard, ProgressCard, Modal, Input, Select, DataTable, Badge, Timeline.

13. Testing and Quality

Level Scope

Unit Auth, permission guards, progress and attendance calculations, recruitment logic

Integration Login, application approval, student creation, attendance, assessment updates, authorization boundaries

E2E (Playwright) Application → approval → student login → dashboard; admin login → create batch → record attendance → update progress

Non-functional Lighthouse (performance, accessibility, SEO), dependency/vulnerability scanning, basic penetration checklist (OWASP Top 10)

Acceptance Each FR-ID has at least one test case; traceability table maintained in /docs/requirements

CI (GitHub Actions): lint → type-check → tests → build → deploy preview → production on approval.

14. Release Plan

Phase Deliverables

1. Public website Home, About, Trainer, Programs, Recruitment, Wall of Fame, Gallery, News, Blog, Contact, Join form, legal pages

2. Authentication Registration flow, login, password reset, roles, profile

3. Student portal Dashboard, attendance, progress, tests, notifications, schedule, targets

4. Trainer/Admin Students, batches, attendance, assessments, progress, notifications, audit log

5. CMS Blog moderation, gallery, events, wall of fame, recruitment updates, testimonials

6. Advanced Reports/PDF, push notifications, PWA, analytics, advanced recruitment tracking

15. Risks and Open Questions

Risk / question Mitigation / action

Handling minors' data and photos Guardian consent, separate media consent, legal review of Indian data-protection obligations

Stale recruitment info misleading students Admin-editable data, "last updated" stamp, always link to the official notification

Inflated or unverifiable statistics Show only substantiated numbers

Scope creep Phase gating; V1 exclusions in §1.4

Slow site from heavy media Performance budget, compressed video, CDN

Single admin dependency Multiple admin accounts with 2FA, documented recovery

Backup never tested Scheduled restore drills

Open questions for the institute: training location(s) and timings; batch structure; whether Bengali content is needed at launch; approximate student count; any fee-based programs in future; existing photos/videos available; who will moderate content; domain name and hosting budget; hosting/data residency preference.

16. Appendix: Traceability Summary

Original requirement SRS sections

Membership application form §3.2

Student profile, notifications, growth tracking §3.4, §3.5, §3.8

Wall of Fame, Gallery, News, Contact §3.1, §3.10 (content)

Admin dashboard: trainers connect with students and see growth §3.9, §3.10

Student blog for motivating journeys §3.11

Modern, animated, energetic yet professional design §12

Secure authentication and full security §3.3, §6

Tech stack, architecture, folder structure §7, §11